# Service Request Tracker

Тестове завдання (backend) · C# / .NET · Middle–Senior

**Час на виконання:** орієнтовно один робочий день. **Стек:** C#, ASP.NET Core, PostgreSQL, RabbitMQ, Docker Compose.

Привіт! Дякуємо, що знайшли час на це завдання. Нижче — опис невеликого сервісу, який ми просимо вас зробити.

## 1. Контекст

Ми розвиваємо систему, у якій клієнти повідомляють про несправності обладнання, а технічні спеціалісти ними займаються. Вам потрібно написати невеликий backend-сервіс: він приймає такі заявки на обслуговування, відстежує їхній статус і повідомляє решті системи, коли із заявкою щось відбувається.

Завдання навмисно невелике. Нам важливіше побачити зрозуміле робоче рішення, у якому ви можете пояснити кожен вибір, ніж щось велике.

## 2. Що потрібно зробити

Єдина сутність — `ServiceRequest`.

| Поле | Тип | Правила |
|---|---|---|
| `Id` | `Guid` | Генерується сервісом. |
| `Title` | `string` | Обов’язкове, до 200 символів. |
| `Description` | `string` | Необов’язкове, до 2000 символів. |
| `Priority` | `enum` | Low, Normal або High. За замовчуванням Normal. |
| `Status` | `enum` | New, InProgress, Done або Cancelled. Нова заявка завжди має статус New. |
| `CreatedAt`, `UpdatedAt` | `timestamp` | Встановлює сервіс, у UTC. |

### API endpoints

| Метод | Маршрут | Поведінка |
|---|---|---|
| `POST` | `/api/service-requests` | Створює заявку. Повертає 201 зі створеним ресурсом. |
| `GET` | `/api/service-requests/{id}` | Повертає одну заявку або 404. |
| `GET` | `/api/service-requests` | Повертає сторінку заявок. Необов’язковий фільтр за статусом. Підтримує page і pageSize. |
| `PATCH` | `/api/service-requests/{id}/status` | Змінює статус за правилами, наведеними нижче. |

Якщо дані не пройшли валідацію, повертайте 400, а якщо заявки з таким id немає — 404. Заборонену зміну статусу відхиляйте з кодом 409. Тіло помилки — у форматі ProblemDetails.

### Правила статусів

- Звичайний потік: New → InProgress → Done.

- Скасовану заявку не можна перевести в інший статус, окрім New.
- Заявка зі статусом Done є остаточною.
- Будь-яка інша зміна заборонена.

### Обмін повідомленнями

Щоразу, коли заявку створено або в неї змінився статус, сервіс публікує одне повідомлення в RabbitMQ. У ньому мають бути id заявки, старий статус (порожній під час створення), новий статус, пріоритет і час зміни.

Ці повідомлення читає споживач (consumer); він може працювати в тому самому процесі. Споживач має зберігати подію та логувати її, якщо вона має високий пріоритет або завершена. Збережені події записуються в окрему таблицю в тій самій базі PostgreSQL.

Повідомлення не мають губитися, навіть якщо RabbitMQ тимчасово недоступний або споживач завершився помилкою під час обробки.

## 3. Технічні вимоги

- Використовуйте .NET 8 або новіший, PostgreSQL як єдину базу даних і RabbitMQ як єдиний брокер повідомлень.
- `docker compose up` має піднімати все: API, PostgreSQL і RabbitMQ. Схема бази даних має створюватися автоматично, без ручних кроків.
- Налаштування беріть зі змінних середовища або appsettings, а не зашивайте їх у коді.
- Напишіть тести для частин, які вважаєте найважливішими. Повного покриття ми не очікуємо.

## 4. Що надіслати нам

- Git-репозиторій (або zip разом із папкою .git) із зрозумілою історією комітів.
- `README.md` з описом: як запустити й протестувати рішення, які рішення та припущення ви ухвалили і що зробили б далі, якби мали більше часу. Обсяг — одна сторінка.
- Перелік усього, що здалося вам незрозумілим або суперечливим у цьому документі, і що ви вирішили з цього приводу.